Alt+F1, θα διαπιστώσετε πως θα ανοίξει η τεκμηρίωση της κλάσης String[7]. Εκεί μπορείτε να βρείτε τεκμηρίωση για κάθε μέλος της κλάσης.

Η συγγραφή της τεκμηρίωσης κάθε κλάσης βασίζεται σε ειδικά σχόλια, τα επονομαζόμενα σχόλια τεκμηρίωσης (doc comments). Σχόλιο τεκμηρίωσης θεωρείται οποιοδήποτε κείμενο βρίσκεται μεταξύ των χαρακτήρων /** που σηματοδοτούν την αρχή του σχολίου και τους χαρακτήρες */ που σηματοδοτούν το τέλος του. Επομένως, υπάρχει διαφορά από τα απλά σχόλια που περιλαμβάνονται μεταξύ των χαρακτήρων /* και */. Τα μεν σχόλια τεκμηρίωσης απαιτούν δύο αστερίσκους μετά την πλάγια κάθετο ενώ τα απλά ένα. Τα απλά σχόλια δεν λαμβάνονται υπόψη από το JavaDoc.

Τα σχόλια τεκμηρίωσης τοποθετούνται πριν τη δήλωση οποιασδήποτε κλάσης ή οποιουδήποτε μέλους κλάσης. Ωστόσο, υπάρχει η δυνατότητα να παράγουμε τεκμηρίωση που αφορά ένα πακέτο, δηλαδή ένα σύνολο κλάσεων ή και για ένα σύνολο πακέτων [5].

Η τεκμηρίωση JavaDoc αφορά κυρίως τα δημόσια μέλη των κλάσεων. Παρότι το εργαλείο προσφέρει τη δυνατότητα τεκμηρίωσης και των μη δημόσιων μελών, η χρησιμότητά της αμφισβητείται από τη μεγάλη πλειοψηφία των επαγγελματιών του χώρου. Αντί της τεκμηρίωσης JavaDoc για τα μη δημόσια μέλη είναι προτιμητέα τα απλά σχόλια με βάση το σκεπτικό πως τα δημόσια μέλη θα χρησιμοποιηθούν από τρίτους επομένως μόνο η τεκμηρίωση των δημόσιων μελών έχει ενδιαφέρον προς τρίτους.

Κατά τη συγγραφή τεκμηρίωσης μπορούμε να χρησιμοποιήσουμε μια σειρά από tags για να προσδιορίσουμε τον τύπο της πληροφορίας που περιγράφουμε. Πριν τη χρήση των tags, μπορεί να προστεθεί μια περιγραφή του υπό τεκμηρίωση αντικειμένου. Παρουσιάζουμε εδώ τα πολύ βασικά tags. Πλήρης λίστα των διαθέσιμων tags μπορεί να βρεθεί στην [6].

Βασικά tags
author
see
param
return
throws

Πίνακας 8.1 *Βασικά tags JavaDoc*

Στο tag author αναφέρεται προφανώς το όνομα του συγγραφέα ή των συγγραφέων του κώδικα. Το tag εισάγεται αυτόματα από τα περισσότερα IDE και από το NetBeans. Ωστόσο, δεν συμπεριλαμβάνεται στο παραγόμενο JavaDoc.

Το tag see χρησιμοποιείται όταν η τεκμηρίωση αναφέρεται σε κάποια άλλη κλάση ή μέλος κλάσης από αυτήν στην οποία ανήκει το tag. Το tag δημιουργεί έναν σύνδεσμο στο αναφερόμενο αντικείμενο.

Το tag param χρησιμοποιείται για την περιγραφή παραμέτρου συνάρτησης.

Το tag return για την περιγραφή της επιστρεφόμενης τιμής από συνάρτηση.

Τέλος, το tag throws για την περιγραφή τυχόν εξαιρέσεων που μπορεί να παράγονται από συνάρτηση.

Στη συνέχεια στον κώδικα 8.6 παρουσιάζουμε τον κώδικα του παιχνιδιού ζαριών με απλά σχόλια και με σχόλια τεκμηρίωσης ανάλογα με τον χαρακτηρισμό προσπέλασης συναρτήσεων και μεταβλητών.

```
import java.util.Random;

  /**
   *Η κλάση προσομοιώνει το ακόλουθο παιχνίδι ζαριών. Τρεις
   *φίλοι, ο John, ο Jim και ο Jack παίζουν ένα παιχνίδι
   *ζαριών. Σύμφωνα με τους κανόνες του παιχνιδιού, κατά τη
   *διάρκεια μιας παρτίδας, οι παίκτες ρίχνουν εναλλάξ το ζάρι
   *10 φορές και μετράνε πόσες φορές έφερε ο καθένας τους άρτια
```

[7] Αν με Alt+F1 δεν ανοίξει αυτόματα η τεκμηρίωση της κλάσης String, μεταβείτε στο NetBeans στο μενού Tools/Java Platforms. Στο παράθυρο που θα ανοίξει, επιλέξτε το (Default) JDK. Σύμφωνα με τις οδηγίες αυτού του βιβλίου πρέπει να είναι το JDK 17. Από τις καρτέλες δεξιά, επιλέξτε την καρτέλα JavaDoc και μεταβάλλετε το URL https://docs.oracle.com/en/Java/Javase/16/docs/api σε https://docs.oracle.com/en/Java/Javase/16/docs/api/Java.base.

```
    *ζαριά.
    * Νικητής είναι εκείνος που έφερε τις περισσότερες άρτιες
    *ζαριές. Αν όμως 2 ή περισσότεροι παίκτες έχουν
    *ισοβαθμήσει, η παρτίδα επαναλαμβάνεται για όλους.
    * Η επανάληψη συνεχίζεται μέχρις ότου να επικρατήσει ένας
    *μοναδικός νικητής. Οι συγκεκριμένοι παίκτες όμως έχουν τις
    *ιδιομορφίες τους. Ο John είναι ένας τίμιος παίκτης με την
    *έννοια πως κάθε φορά που πετάει το ζάρι έχει την ίδια
    *πιθανότητα να φέρει οποιοδήποτε αποτέλεσμα από 1 έως 6. Από
    *την άλλη μεριά, ο Jim είναι τυχερός καθώς έχει αποδειχτεί
    *πως όταν πετάει το ζάρι έχει 15% πιθανότητα για κάθε ένα
    *από τα αποτελέσματα 1, 3 και 5, 18% για κάθε ένα από τα
    *αποτελέσματα 2 ή 4 και 19% για να φέρει 6. Επομένως, ο Jim
    *φέρνει άρτια ζαριά με πιθανότητα 55% και περιττή ζαριά με
    *πιθανότητα 45%. Αντίθετα, ο Jack είναι άτυχος καθώς φέρνει
    *2, 4, 6 με πιθανότητα 15% για κάθε ένα από αυτά τα
    *αποτελέσματα, ενώ 1 και 3 φέρνει με πιθανότητα 18% και 5
    *φέρνει με πιθανότητα 19%. Επομένως, ο Jack φέρνει άρτια
    *ζαριά με πιθανότητα 45% και περιττή ζαριά * με πιθανότητα
    *55%.
    *
    * @author Lefteris Moussiades
    */
public class ADiceGame {

    /**
     * Χρησιμοποιείται απο τον έλεγχο ισότητας πραγματικών
     *
     * @see approximateEquals
     */
    static final double EPSILON = 0.00001;
    private static final Random gen = new Random();
    static final double[] fair = {1/6d, 1/6d, 1/6d, 1/6d, 1/6d, 1/6d};
    static final double[] lucky = {15d/100, 18d/100, 15d/100, 18d/100, 15d/100,
19d/100};
    static final double[] unlucky = {18d/100, 15d/100, 18d/100, 15d/100, 19d/100,
15d/100};
    private static final int JOHN = 0;
    private static final int JIM = 1;
    private static final int JACK = 2;

    /**
     * Συγκρίνει προσεγγιστικά δύο πραγματικούς τύπου double για έλεγχο
     * ισότητας
     *
     * @param d1
     * @param d2
     * @return Επιστρέφει true αν η απόλυτη τιμή της διαφορας d1-d2 είναι
     * μικρότερη απο την τιμή της EPSILON
     * @see EPSILON
     */
    public static boolean approximateEquals(double d1, double d2) {
        return Math.abs(d1 - d2) < EPSILON;
    }

    /**
     * Υπολογίζει το άθροισμα των στοιχείων της παραμέτρου array
     *
     * @param array
     * @return Το άθροισμα των στοιχείων της array
     */
```

```
    public static double sum(double[] array) {
        double sum = 0;
        for (double d : array) {
            sum += d;
        }
        return sum;
    }

    /**
     * Προσομοιώνει τη ρίψη ζαριού
     *
     * @param distribution Πίνακας πραγματικών. Κατά την κλήση της μεθόδου, η
     * distribution πρέπει να περιλαμβάνει 6 στοιχεία με άθροισμα ίσο με 1.
     * @return Έναν αριθμό απο 1 έως 6 ανάλογα με την πιθανότητα που δίνεται
     * για κάθε τιμή μέσω της παραμέτρου distribution
     * @throws RuntimeException
     */
    public static int roll(double[] distribution) {
        if (distribution.length != 6 || !approximateEquals(sum(distribution),
1d)) {
            throw new RuntimeException();
        }
        double value = gen.nextDouble();
        double sum = 0;
        int rslt = 1;
        for (double percentage : distribution) {
            sum += percentage;
            if (value <= sum) {
                return rslt;
            }
            rslt++;
        }
        return 0;
    }

    /**
     * Εντοπίζει τη μέγιστη τιμή του πίνακα array
     *
     * @param array Ο πίνακας του οποίου το μέγιστο στοιχείο θέλουμε να
     * βρούμε
     * @return Επιστρέφει τη θέση της πρώτης απο τις μέγιστες τιμές του
     * πίνακα
     * array
     */
    public static int max(int[] array) {
        int idx = 0;
        int max = array[0];
        for (int i = 1; i < array.length; i++) {
            if (array[i] > max) {
                max = array[i];
                idx = i;
            }
        }
        return idx;
    }

    /**
     * Υπολογίζει πόσες φορές υπάρχει μια τιμή σε έναν πίνακα
     *
     * @param array Ο πίνακας στον οποίο θα μετρήσουμε πόσες φορές υπάρχει η
     * value
```

```
 * @param value Η τιμή που θα μετρήσουμε πόσες φορές υπάρχει στον array
 * @return Επιστρέφει το πλήθος των στοιχείων με τιμή ίση με value στον
 * πίνακα array
 */
public static int cntOccurances(int[] array, int value) {
    int cnt = 0;
    for (int i = 0; i < array.length; i++) {
        if (value == array[i]) {
            cnt++;
        }
    }
    return cnt;
}

private static boolean withdraw(int[] score) {
    return cntOccurances(score, score[max(score)]) > 1;
}

/*
Η aLot  λαμβάνει ως παράμετρο τον αριθμό των ρίψεων ανα παρτίδα και
παίκτη.
Δηλώνει τον πίνακα score όπου αποθηκεύονται οι ζαριές με άρτιο αποτέλεσμα
για κάθε παίκτη κατά τη διάρκεια της παρτίδας.
Προκειμένου να αντιστοιχίζεται κάθε παίκτης ευκολότερα, με περισσότερη
ασφάλεια και να είναι το πρόγραμμα ευανάγνωστο,
έχουν δηλωθεί οι σταθερές JOHN, JIM και JACK ως 0, 1 και 2. Έτσι γίνεται
μια αντιστοιχία μεταξύ των θέσεων στον πίνακα των σκορ
με τα ονόματα των παικτών.
Μέσα στον βρόγχο for της aLot κάθε παίκτης ρίχνει τη ζαριά του με τη
βοήθεια της roll και αν το αποτέλεσμα είναι άρτιο ενημερώνεται
το σκορ του.
*/
private static int[] aLot(int noOfRolls) {
    int[] score = new int[3];
    for (int i = 0; i < noOfRolls; i++) {
        int rslt = roll(fair);
        if (rslt % 2 == 0) {
            score[JOHN]++;
        }

        rslt = roll(lucky);
        if (rslt % 2 == 0) {
            score[JIM]++;
        }

        rslt = roll(unlucky);
        if (rslt % 2 == 0) {
            score[JACK]++;
        }
    }
    return score;
}

/*
Η game υλοποιεί ένα παιχνίδι με συγκεκριμένα χαρακτηριστικά παρτίδας.
Έτσι δηλώνει ως τοπική σταθερά τον αριθμό ρίψεων κάθε παρτίδας.
Στη συνέχεια, σύμφωνα με τον σχεδιασμό καλεί την aLot μέσα σε
επαναληπτική διαδικασία που ελέγχεται από την withdraw.
Σε κάθε κλήση, η aLot επιστρέφει έναν πίνακα με τα σκορ των παικτών.
Τα σκορ ελέγχει η withdraw. Αν η withdraw δεν ανιχνεύσει ισοπαλία,
τερματίζεται ο βρόχος do και επιστρέφεται η θέση του πίνακα
```

```
    των σκορ με τη μεγαλύτερη τιμή. Εκεί βρίσκεται ο νικητής.
     */
    private static int game() {
        final int NOOFROLLS = 10;
        int[] score;

        do {
            score = aLot(NOOFROLLS);
        } while (withdraw(score));
        return max(score);
    }

    private static String winnerName(int winnerId) {

        switch (winnerId) {
            default:
                throw new RuntimeException();
            case JOHN:
                return "John";
            case JIM:
                return "Jim";
            case JACK:
                return "Jack";
        }
    }

    public static void main(String[] args) {
        int winnerId = game();
        System.out.println("winner is " + winnerName(winnerId));

    }
}
```

Κώδικας 8.6 *Ο κώδικας του παιχνιδιού ζαριών σχολιασμένος*

Μετά την ολοκλήρωση σχολιασμού του κώδικα, μεταβείτε στην περιοχή Projects, κάντε δεξί κλικ στο όνομα του project στο οποίο έχετε σχολιάσει τις κλάσεις και επιλέξτε Generate Javadoc. Η τεκμηρίωση του κώδικα σε μορφή HTML θα ανοίξει στον φυλλομετρητή του συστήματος. Επιπλέον, με δεξί κλικ επάνω σε κάθε κλάση και επιλογή Tools/Analyze Javadoc μπορείτε να λάβετε μια αναφορά για τυχόν παραλείψεις στην τεκμηρίωση της κλάσης.

8.7 Απολαθοποίηση

Η απολαθοποίηση (debugging) είναι μια σημαντική λειτουργία χρήσιμη τόσο κατά το στάδιο της υλοποίησης όσο και κατά το στάδιο του ελέγχου όπου ενδέχεται να προκύψουν προβλήματα και να κληθούν οι προγραμματιστές να εντοπίσουν την εστία τους μέσα στον κώδικα και να τα θεραπεύσουν. Η απολαθοποίηση παρέχει τη δυνατότητα να εκτελέσουμε τον κώδικα ελεγχόμενα, βήμα προς βήμα και σε κάθε βήμα να ελέγχουμε τις τιμές των μεταβλητών και των εκφράσεων.
Θα δούμε τη λειτουργία του απολαθοποιητή στο NetBeans με ένα παράδειγμα.

```
import gr.ihu.cs.lmous.OOJ.k7.v1.StaticFunctions;
import java.util.Arrays;

public class Debugging {

    static int[] resize(int[] in, int newSize) {
        int[] rVal=new int[newSize];
        for (int i=0; i<in.length && i<rVal.length; i++)
            rVal[i]=in[i];
        return rVal;
```

```
    }

    static int[] createSet(int... elements) {
        int[] set=new int[elements.length];
        int setIdx=0;
        for (int element:elements) {
            int idx=StaticFunctions.sequentialSearch(set, element);
            if (idx<0)
                set[setIdx++]=element;
        }
        resize(set, setIdx);
        return set;
    }

    static int[] union(int[] set1, int[] set2) {
        int[] union=new int[set1.length+set2.length];
        System.arraycopy(set1, 0, union, 0, set1.length);
        System.arraycopy(set2, 0, union, set1.length, set2.length);
        return createSet(union);
    }

    public static void main(String[] args) {
        int[] set1=createSet(2,4,6,3), set2=createSet(3,5,7,2);
        int[] union=union(set1, set2);
        System.out.println(Arrays.toString(union));
    }
}
```

Κώδικας 8.7 *Κώδικας προς απολαθοποίηση*

Ο κώδικας 8.7 είναι μια εφαρμογή διαχείρισης συνόλων ακέραιων. Κάθε στοιχείο ενός συνόλου είναι μοναδικό, δηλαδή σε ένα σύνολο δεν μπορεί να υπάρχει δύο η περισσότερες φορές το ίδιο στοιχείο. Η συνάρτηση createSet λαμβάνει ως παράμετρο μια σειρά ακέραιων και επιστρέφει ένα σύνολο ακέραιων με τη μορφή πίνακα. Για να το πετύχει αυτό, εισάγει στον πίνακα που επιστρέφει μια τιμή από τις πραγματικές παραμέτρους μόνο εφόσον αυτή δεν υπάρχει ήδη. Μάλιστα πριν επιστρέψει καλεί την resize ώστε να ρυθμίσει το μέγεθος του πίνακα που επιστρέφει ανάλογα με το πλήθος των στοιχείων του συνόλου. Η συνάρτηση union λαμβάνει ως παραμέτρους δύο πίνακες που αντιπροσωπεύουν σύνολα και επιστρέφει την ένωσή τους. Στην main δημιουργούμε δύο σύνολα, υπολογίσουμε την ένωσή τους και την τυπώνουμε.
Αν τρέξετε τον κώδικα θα δείτε πώς η έξοδός του είναι

```
[2, 4, 6, 3, 5, 7, 0, 0]
```

Το σύνολο-ένωση όμως που αναμένουμε είναι το [2, 4, 6, 3, 5, 7]. Επομένως κάποιο λάθος υπάρχει στον κώδικα. Μπορείτε να το βρείτε; Αξίζει να προσπαθήσετε καταρχάς απλώς διαβάζοντας τον κώδικα. Αν ωστόσο δεν τα καταφέρετε, ο απολαθοποιητής είναι εδώ για να σας βοηθήσει.
Μεταβείτε στο NetBeans και δημιουργήστε την κλάση Debugging. Στην πρώτη γραμμή της main, κάντε κλικ στην γκρίζα κάθετη αριθμημένη λωρίδα στα αριστερά του κώδικα. Η γραμμή θα αποκτήσει ένα ροζ φόντο όπως φαίνεται στην εικόνα 8.1.

```
39      public static void main(String[] args) {
            int[] set1=createSet(2,4,6,3), set2=createSet(3,5,7,2);
41          int[] union=union(set1, set2);
42          System.out.println(Arrays.toString(union));
43      }
```

Εικόνα 8.1 *Breakpoint στην πρώτη γραμμή της main*

Έχετε τοποθετήσει ένα Breakpoint (σημείο διακοπής). Μπορείτε τώρα να εκτελέσετε τον κώδικα κατά τρόπο ώστε να διακοπεί η ροή του όταν συναντήσει το Breakpoint. Ανάλογα με τις ανάγκες σας μπορείτε να τοποθετήσετε και άλλα Breakpoint.
Μεταβείτε στην κλάση στο πλαίσιο Projects. Με δεξί κλικ πάνω στην κλάση ανοίγει ένα μενού, επιλέξτε Debug File. Θα μεταβείτε στην κατάσταση που δείχνει η εικόνα 8.2.

```
39      public static void main(String[] args) {
            int[] set1=createSet(2,4,6,3), set2=createSet(3,5,7,2);
41          int[] union=union(set1, set2);
42          System.out.println(Arrays.toString(union));
43      }
```

Εικόνα 8.2 *Διακοπή της ροής στο Breakpoint*

Η εφαρμογή εκτελείται αλλά έχει σταματήσει στην πρώτη γραμμή της main, εκεί δηλαδή που συνάντησε το πρώτο Breakpoint. Στη συνέχεια έχουμε έλεγχο στη ροή του προγράμματος με τη βοήθεια των εντολών προς τον απολαθοποιητή. Στη γραμμή εργαλείων, κάτω από το κεντρικό μενού βλέπετε την εργαλειοθήκη με τις διαθέσιμες εντολές προς τον απολαθοποιητή.



Εικόνα 8.3 *Η εργαλειοθήκη του απολαθοποιητή.*

Με αναφορά στην αρίθμηση της εικόνας 8.3, οι διαθέσιμες εντολές προς τον απολαθοποιητή έχουν ως εξής:

Finish Debugger Session: Τερματίζεται η λειτουργία του απολαθοποιητή.

Pause: Προσωρινή παύση της εκτέλεσης του υπό απολαθοποίηση προγράμματος.

Continue: Συνέχιση της εκτέλεσης του υπό απολαθοποίηση προγράμματος.

Step Over: Εκτέλεση της τρέχουσας συνάρτησης.

Step Over Expression: Εκτέλεση της τρέχουσας έκφρασης.

Step Into: Είσοδος στην τρέχουσα συνάρτηση.

Step Out: Έξοδος από την τρέχουσα συνάρτηση.

Run to cursor: Εκτέλεση του προγράμματος μέχρι του σημείου που έχουμε τοποθετήσει τον cursor στον κώδικα.

Καθώς το πρόγραμμα έχει μπει σε κατάσταση απολαθοποίησης, στο πλαίσιο κάτω από τον κώδικα, εκεί όπου βρίσκεται το παράθυρο Output, έχουν ανοίξει δύο ακόμη παράθυρα, το Variables και το Breakpoints. Στο παράθυρο Breakpoints μπορούμε να βλέπουμε και να διαχειριζόμαστε τα Breakpoints που τοποθετούμε στον κώδικα και στο παράθυρο Variables βλέπουμε τις μεταβλητές που είναι εντός εμβέλειας ανάλογα με το σημείο του προγράμματος στο οποίο βρισκόμαστε. Για παράδειγμα, στην τρέχουσα κατάσταση, μπορούμε να δούμε πως η παράμετρος args έχει μήκος 0, κλήθηκε επομένως η εφαρμογή χωρίς παραμέτρους της main.

Πατήστε το εικονίδιο Step Over. Η πρώτη γραμμή έχει τρέξει. Ο απολαθοποιητής περιμένει στην επόμενη γραμμή, ενώ παράλληλα στο παράθυρο Variables έχουν εμφανιστεί οι μεταβλητές set1 και set2.

Μπορούμε να ανοίξουμε τις μεταβλητές για να δούμε τα περιεχόμενά τους. Είναι αυτά που αναμένουμε; Πράγματι, μια πρώτη επιθεώρηση μας δίνει τις αναμενόμενες τιμές τόσο για την set1 όσο και για την set2. Δεδομένου ότι έχουμε ένα πολύ μικρό πρόγραμμα και είδαμε πως η πρώτη γραμμή εκτελείται με το σωστό αποτέλεσμα, λογικό είναι να υποθέσουμε πως κάτι δεν πάει καλά στη δεύτερη γραμμή όπου εκτελείται η union. Για αυτόν τον λόγο θα επιθεωρήσουμε πιο στενά την union.

Πατήστε Step Into, η ροή του κώδικα θα σας μεταφέρει στην πρώτη γραμμή της union. Αν προχωρήσουμε βήμα προς βήμα με Step Over μέχρι την τελευταία γραμμή της union, δεν θα συναντήσουμε κάποιο πρόβλημα. Επομένως θα υποπτευθούμε πως κάτι μπορεί να συμβαίνει στην τελευταία γραμμή της union

όπου καλείται η createSet. Μα την createSet την έχουμε ήδη δοκιμάσει και φαίνεται να πήγε καλά. Ας μπούμε καλύτερα μέσα με Step Into να δούμε με μεγαλύτερη λεπτομέρεια.

Με Step Over στις γραμμές της create και έλεγχο των διαθέσιμων μεταβλητών διαπιστώνουμε πως το πρόβλημα είναι πως το μέγεθος του πίνακα set δεν μεταβάλλεται μετά την εκτέλεση της resize. Ενώ ο πίνακας set δημιουργήθηκε αρχικά με μέγεθος το άθροισμα των μεγεθών των δύο συνόλων που ενώνονται, μετά την resize το μέγεθός του αναμένεται μειωμένο κατά δύο καθώς τα στοιχεία 2 και 3 υπάρχουν και στα δύο σύνολα.

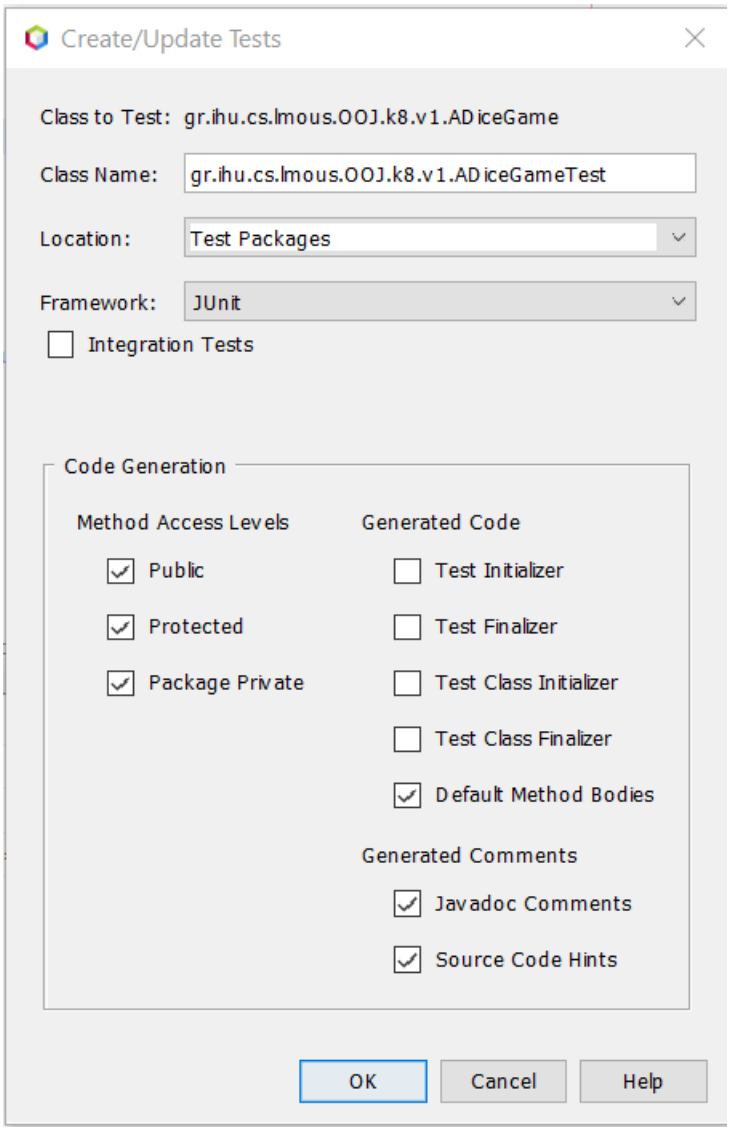
Μετά τη διαπίστωση αυτή, παρατηρήστε πως η resize δεν μεταβάλλει το μήκος του πίνακα που περνάει ως παράμετρος σε αυτήν. Αντίθετα, δημιουργεί και επιστρέφει ένα αντίγραφο της παραμέτρου με το σωστό μήκος. Εφόσον επιθυμούμε να αλλάξουμε το μέγεθος του πίνακα set, θα πρέπει να εκχωρήσουμε την τιμή που επιστρέφει η resize στη μεταβλητή set. Επομένως αλλάξτε τη γραμμή resize(set, setIdx) σε set=resize(set, setIdx), τερματίστε τον απολαθοποιητή και ξανατρέξτε την κλάση. Θα διαπιστώσετε πως η ένωση των συνόλων εμφανίζεται σωστά αυτήν τη φορά.

8.8 Έλεγχος

Ο έλεγχος (test) αποτελεί ένα απαραίτητο στοιχείο της ανάπτυξης. Είναι σημαντικό να κατανοήσουμε πως ο έλεγχος δεν γίνεται αποκλειστικά μετά την ολοκλήρωση της υλοποίησης. Αντίθετα, ξεκινά παράλληλα με την υλοποίηση και διεξάγεται καθόλη τη διάρκειά της.

Ο έλεγχος επιτυγχάνεται με τη συγγραφή και εκτέλεση κατάλληλων συναρτήσεων. Θα πρέπει δηλαδή για κάθε μια μη ιδιωτική συνάρτηση της εφαρμογής να αναπτύξουμε μια αντίστοιχη συνάρτηση ελέγχου. Θα δούμε περισσότερες λεπτομέρειες στην πράξη αναπτύσσοντας κώδικα ελέγχου για την κλάση ADiceGame.

Στο παράθυρο Projects, μεταβείτε στην 1η έκδοση της κλάσης και πατήστε δεξί κλικ. Στο παράθυρο που θα ανοίξει επιλέξτε Tools/Create Tests. Θα ανοίξει το παράθυρο που φαίνεται στην εικόνα 8.4.



Εικόνα 8.4 *Παράμετροι δημιουργίας test*

Στην ενότητα Code Generation, θέστε τις επιλογές όπως φαίνονται στην εικόνα 8.4 και πατήστε OK. Θα διαπιστώσετε πως δημιουργήθηκαν δύο νέοι φάκελοι κάτω από το project. Οι φάκελοι Test Packages και Test Libraries. Κάντε δεξί κλικ στον φάκελο Test Libraries και στο μενού που θα ανοίξει επιλέξτε Add Library. Θα ανοίξει τότε ένα παράθυρο με τις διαθέσιμες βιβλιοθήκες. Εντοπίστε το Junit 4.12, επιλέξτε το και πατήστε Add Library. Ανοίξτε τον φάκελο Test Libraries και διαγράψτε (remove) τις υπόλοιπες βιβλιοθήκες πλην της Junit 4.12 και της Hamcrest 1.3. Αν δεν βλέπετε την Hamcrest, προσθέστε την όπως προσθέσαμε την Junit 4.12.

Μεταβείτε στον φάκελο Test Packages, εντοπίστε το ADiceGameTest και διαγράψτε το. Τέλος μεταβείτε και πάλι στην 1η έκδοση της κλάσης, πατήστε δεξί κλικ και δημιουργήστε εκ νέου το ADiceGameTest. Η κλάση θα ανοίξει αυτόματα στην περιοχή του κώδικα. Περιηγηθείτε στον κώδικα της κλάσης και παρατηρήστε πως για κάθε μη ιδιωτική συνάρτηση της κλάσης AdiceGame, η κλάση AdiceGameTest περιλαμβάνει μια συνάρτηση ελέγχου. Για παράδειγμα, για την συνάρτηση approximateEquals περιλαμβάνει την TestApproximateEquals, για την sum την TestSum, κοκ. Θα πρέπει να διορθώσουμε κατάλληλα τις συναρτήσεις ελέγχου ώστε να επιτελούν το έργο τους.

Αντικαταστήστε την ADiceGameTest που παρήχθη αυτόματα με την ADiceGameTest του κώδικα 8.9 όπου έχουν γίνει οι απαιτούμενες αλλαγές.

```
import java.util.Random;
import org.junit.Test;
import static org.junit.Assert.*;
```

```java
/**
 *
 * @author Lefteris Moussiades <lmous@cs.ihu.gr>
 */
public class ADiceGameTest {

    /**
     * Test of approximateEquals method, of class ADiceGame.
     */
    @Test
    public void testApproximateEquals() {
        System.out.println("approximateEquals");
        double d1 = 0.0;
        double d2 = ADiceGame.EPSILON;
        boolean result = ADiceGame.approximateEquals(d1, d2);
        assertEquals(false, result);
        d2 = ADiceGame.EPSILON / 2;
        assertEquals(true, ADiceGame.approximateEquals(d1, d2));
    }

    /**
     * Test of sum method, of class ADiceGame.
     */
    @Test
    public void testSum() {
        System.out.println("sum");
        double[] array = ADiceGame.fair;
        double result = ADiceGame.sum(array);
        assertEquals(1d, result, ADiceGame.EPSILON);
        array = ADiceGame.lucky;
        result = ADiceGame.sum(array);
        assertEquals(1d, result, ADiceGame.EPSILON);
        array = ADiceGame.unlucky;
        result = ADiceGame.sum(array);
        assertEquals(1d, result, ADiceGame.EPSILON);
    }

    /**
     * Test of roll method, of class ADiceGame.
     */
    @Test
    public void testRoll() {
        System.out.println("roll");
        double[] distribution;
        int sumFair = 0, sumLucky = 0, sumUnlucky = 0, rslt;
        for (int i = 0; i < 10000; i++) {
            rslt = ADiceGame.roll(ADiceGame.fair);
            if (rslt < 1 || rslt > 6) {
                fail("roll result " + rslt);
            }
            sumFair += rslt % 2 == 0 ? 1 : 0;
            rslt = ADiceGame.roll(ADiceGame.lucky);
            if (rslt < 1 || rslt > 6) {
                fail("roll result " + rslt);
            }
            sumLucky += rslt % 2 == 0 ? 1 : 0;
            rslt = ADiceGame.roll(ADiceGame.unlucky);
            if (rslt < 1 || rslt > 6) {
                fail("roll result " + rslt);
            }
            sumUnlucky += rslt % 2 == 0 ? 1 : 0;
```

```
        }
        System.out.println(sumLucky/10000d+" "+sumFair/10000d+"
"+sumUnlucky/10000d);
        assertTrue(sumLucky > sumFair && sumFair > sumUnlucky);
    }

    /**
     * Test of max method, of class ADiceGame.
     */
    public static void shuffle(int[] t) {
        Random r = new Random();

        for (int i = 0; i < t.length; i++) {
            int rIdx = r.nextInt(t.length);
            int tmp = t[i];
            t[i] = t[rIdx];
            t[rIdx] = tmp;
        }
    }

    private int[] constructArray(int initialValue) {
        int[] rVal = new int[10];
        for (int i = 0; i < 10; i++) {
            rVal[i] = initialValue + i;
        }
        shuffle(rVal);
        return rVal;
    }

    @Test
    public void testMax() {
        System.out.println("max");
        int arrayNo = 0, initialValue = 1;
        int[] array;
        while (arrayNo < 1000) {
            array = constructArray(initialValue);
            assertEquals(initialValue + 9, array[ADiceGame.max(array)]);
            initialValue += 10;
            arrayNo++;
        }
    }

    /**
     * Test of cntOccurances method, of class ADiceGame.
     */
    @Test
    public void testCntOccurances() {
        System.out.println("cntOccurances");
        int[] array = {1, 2, 2, 3, 3, 3, 4, 4, 4, 4, 5, 5, 5, 5, 5, 6, 6, 6, 6,
6, 6};
        assertEquals(1, ADiceGame.cntOccurances(array, 1));
        assertEquals(2, ADiceGame.cntOccurances(array, 2));
        assertEquals(3, ADiceGame.cntOccurances(array, 3));
        assertEquals(4, ADiceGame.cntOccurances(array, 4));
        assertEquals(5, ADiceGame.cntOccurances(array, 5));
        assertEquals(6, ADiceGame.cntOccurances(array, 6));
    }

    /**
     * Test of withdraw method, of class ADiceGame.
     */
```

```
    @Test
    public void testWithdraw() {
        System.out.println("withdraw");
        int[] score = {1100,5,1100};
        ADiceGame.withdraw(score);
        assertTrue(ADiceGame.withdraw(score));
        score[2] = 6;
        assertFalse(ADiceGame.withdraw(score));
    }

}
```

Κώδικας 8.9 *Test για την κλάση AdiceGame*

Καταρχάς να επισημάνουμε τον ρόλο μερικών νέων συναρτήσεων που περιλαμβάνονται στην ADiceGameTest και συναντούμε για πρώτη φορά.

assertEquals με δύο παραμέτρους ίδιου τύπου. Ελέγχει τις παραμέτρους και αν δεν είναι ίσες σημειώνει λάθος.

assertEquals με τρεις πραγματικές παραμέτρους. Ελέγχει τις δύο πρώτες παραμέτρους για προσεγγιστική ισότητα. Η τρίτη παράμετρος είναι η ανεκτή διαφορά. Στη δική μας περίπτωση χρησιμοποιούμε το EPSILON της εφαρμογής.

fail σημειώνει λάθος με μήνυμα την πραγματική παράμετρό της.

assertTrue σημειώνει λάθος εφόσον η παράμετρός της είναι ψευδής.

assertFalse σημειώνει λάθος εφόσον η παράμετρός της είναι αληθής.

Ας συζητήσουμε τώρα κάποιες από τις συναρτήσεις της ADiceGameTest ώστε να κατανοήσουμε πώς πρέπει να γράφουμε τις συναρτήσεις ελέγχου.

Η συνάρτηση testSum ελέγχει τη συνάρτηση sum της ADiceGame. Γνωρίζουμε ότι έχουμε τρεις πίνακες πραγματικών δηλωμένους στην ADiceGame, τον fair, τον lucky και τον unlucky και πως το άθροισμα των στοιχείων κάθε ενός από αυτούς τους πίνακες πρέπει να είναι ίσο με 1. Μπορούμε να αναπτύξουμε την testSum βασιζόμενοι σε αυτό το δεδομένο. Πράγματι, αν εξετάσετε τον κώδικα της testSum θα διαπιστώσετε περιλαμβάνει διαδοχικές κλήσεις στην sum, μία για κάθε έναν από τους fair, lucky και unlucky. Λαμβάνει το αποτέλεσμα της κλήσης στη μεταβλητή result την οποία συγκρίνει προσεγγιστικά μέσω της assertEquals με το 1. Αν η assertEquals εντοπίσει απόκλιση, θα σημειώσει λάθος.

Η testRoll καλεί την roll 10.000 φορές για κάθε μία κατανομή, δηλαδή για τους πίνακες fair, lucky και unlucky. Αν παραχθεί αριθμός έξω από το όριο 1 έως 6, καλεί την fail και σημειώνει το λάθος. Στο τέλος του βρόγχου καλείται η assertTrue για να ελέγξει τη συνθήκη sumLucky > sumFair && sumFair > sumUnlucky. Πράγματι, γνωρίζουμε πως η κατανομή lucky φέρνει περισσότερα άρτια αποτελέσματα από ότι η fair και η fair περισσότερα από ότι η unlucky. Μετά από 10.000 ρίψεις, οι πιθανότητες λογικά επαληθεύονται. Σε περίπτωση όμως που δεν επαληθευτούν η assertTrue θα σημειώσει λάθος.

Στην testMax δημιουργούμε 1000 πίνακες με τη βοήθεια της constructArray κατά τρόπο που γνωρίζουμε για κάθε πίνακα την τιμή του μέγιστου στοιχείου του. Προσέξτε πως η constructArray ανακατεύει τα στοιχεία του πίνακα που δημιουργεί έτσι ώστε το στοιχείο με τη μέγιστη τιμή να μην βρίσκεται πάντα στην τελευταία θέση του πίνακα. Στη συνέχεια ελέγχουμε αν η τιμή που επιστρέφει η max ανταποκρίνεται στην αναμενόμενη.

Εφόσον έχετε ακολουθήσει τα βήματα έως εδώ με ακρίβεια, το τεστ θα τρέξει επιτυχώς. Καλό είναι όμως να δείτε τι συμβαίνει σε περίπτωση που υπάρχει κάποια αποτυχία στο τεστ.
Μετατρέψτε προσωρινά τις cntOccurances και withdraw της ADiceGame όπως δείχνει ο κώδικας 8.10.

```
    public static int cntOccurances(int[] array, int value) {
        int cnt = 0;
        for (int i = 0; i < array.length; i++) {
            if (array[value] == array[i]) {
                cnt++;
            }
        }
        return cnt;
    }
```